

Relatório 12 – Pipeline de Dados

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre alguns conceitos importantes sobre projetar *pipeline* de dados automatizados. A tarefa consistia em assistir o minicurso “*Apache Airflow: The Hands-On Guide*” e, como prática, desenvolver uma *pipeline* de dados automatizados apresentando o conhecimento adquirido.

2. Desenvolvimento

Como as aulas do minicurso foi apresentado a construção de uma *pipeline* de dados, onde era realizada a extração de dados de câmbio de moedas, pensei em fazer algo parecido, porém, com ações da B3 (Brasil, Bolsa, Balcão; Bolsa de valores oficial do Brasil).

Inicialmente, encontrei uma biblioteca *Python*, o *yfinance*, ela é uma biblioteca de *open-source* que permite acessar dados financeiros históricos e em tempo real, disponibilizado pelo *Yahoo Finance*. Ela retorna os dados diretamente em *DataFrames*. Depois, descobri que o *Yahoo Finance* bloqueava o *yfinance* e como solução para isso utilizei o *yfinance_cache*, ele atua como um *wrapper* para o *yfinance* e evita as requisições repetidas ao servidor, consequentemente acelera a coleta de dados e reduz o risco de bloqueios por excesso de solicitações. No geral, as funções do *yfinance* e *yfinance_cache* são as mesmas.

- **Obtenção de Dados:** Inicialmente, utilizei o *yfinance_cache* para obter os dados das ações, na qual passei os *tickers* (“id” das ações) dentro de uma lista. Nesta tarefa, ele coleta os dados em um período de 1 dia, pega do dia atual, e armazena em um *DataFrame*.
- **Preparação dos Dados:** Os dados armazenados no *DataFrame* passam por duas tarefas, deveriam ser em paralelo, porém, não soube como fazer isso, uma que vai salvar em um arquivo CSV e outra que vai gerar um resumo do dia para cada ação.
- **Final da Pipeline:** Ao final da *pipeline*, é enviado um email e uma mensagem no Discord, indicando que a *pipeline* foi finalizada, no caso de ter sido finalizada com sucesso.

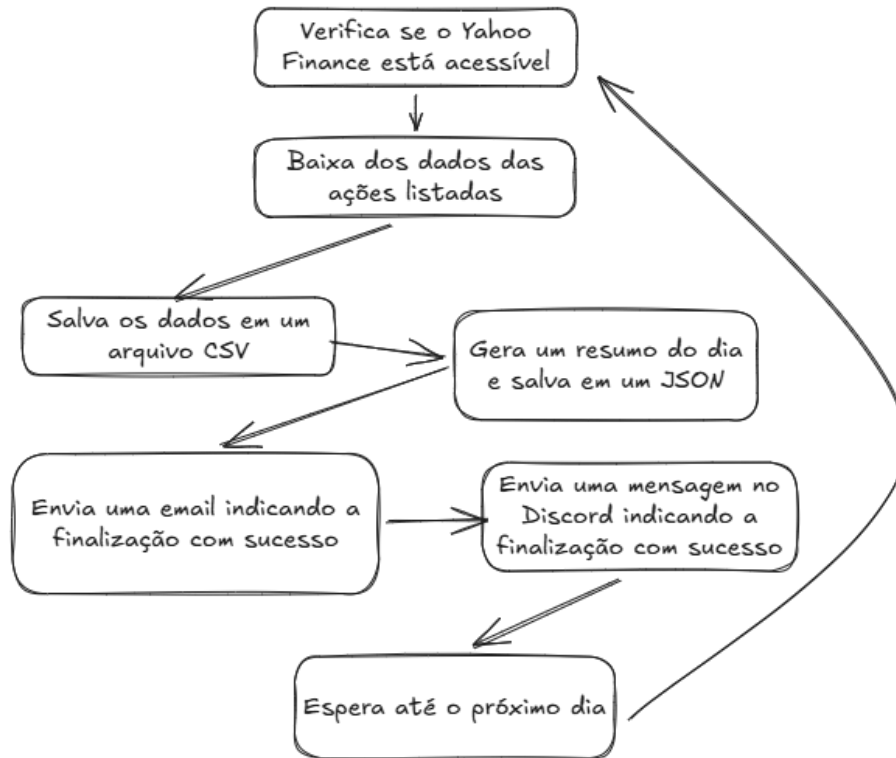
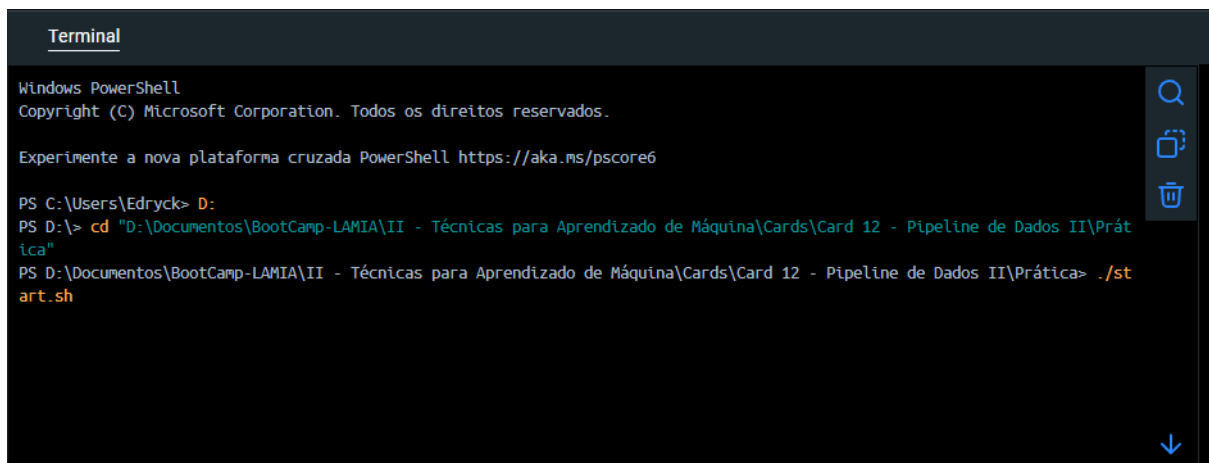


Figura 1. Pipeline da dados B3. (autoria própria)

Usando o comando *docker init* conseguir criar o

A seguir, mostrarei a sequência de ações realizadas no terminal e no Airflow para criação desta *pipeline*.



```
Terminal
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Experimente a nova plataforma cruzada PowerShell https://aka.ms/pscore6

PS C:\Users\Edryck> D:
PS D:\> cd "D:\Documentos\BootCamp-LAMIA\II - Técnicas para Aprendizado de Máquina\Cards\Card 12 - Pipeline de Dados II\Prática"
PS D:\Documentos\BootCamp-LAMIA\II - Técnicas para Aprendizado de Máquina\Cards\Card 12 - Pipeline de Dados II\Prática> ./start.sh
```

Figura 2. Subindo o contêiner. (autoria própria)

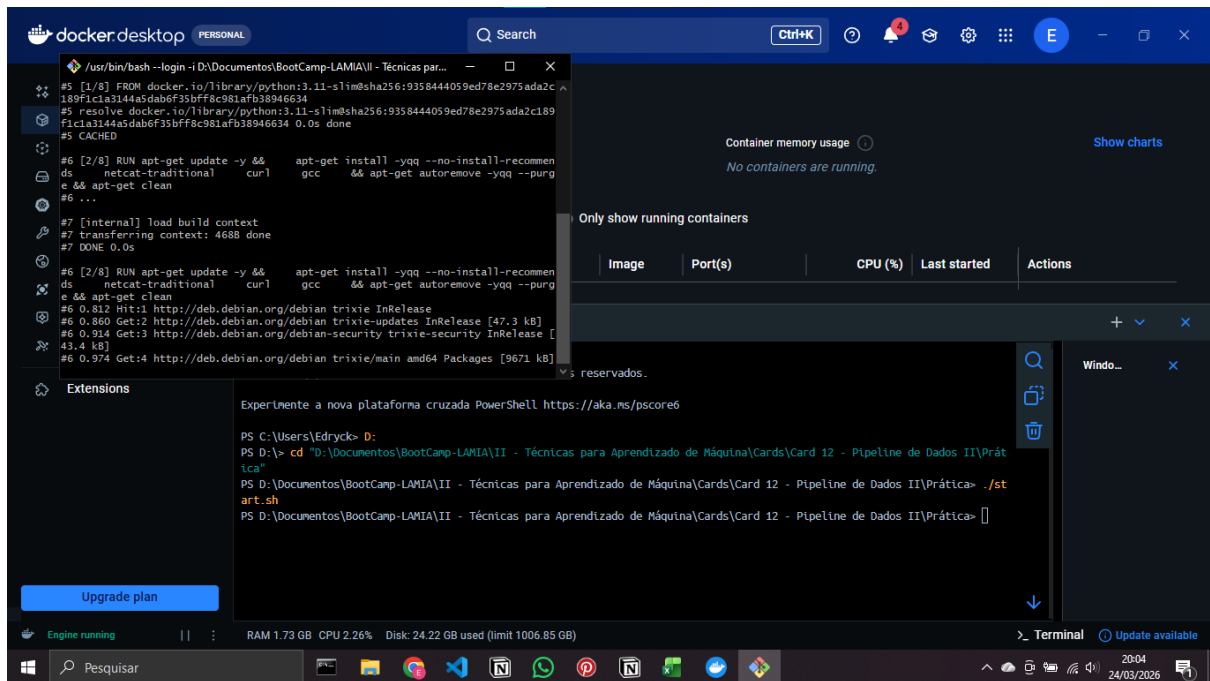


Figura 3. Esperando terminar de subir o contêiner... (autoria própria)

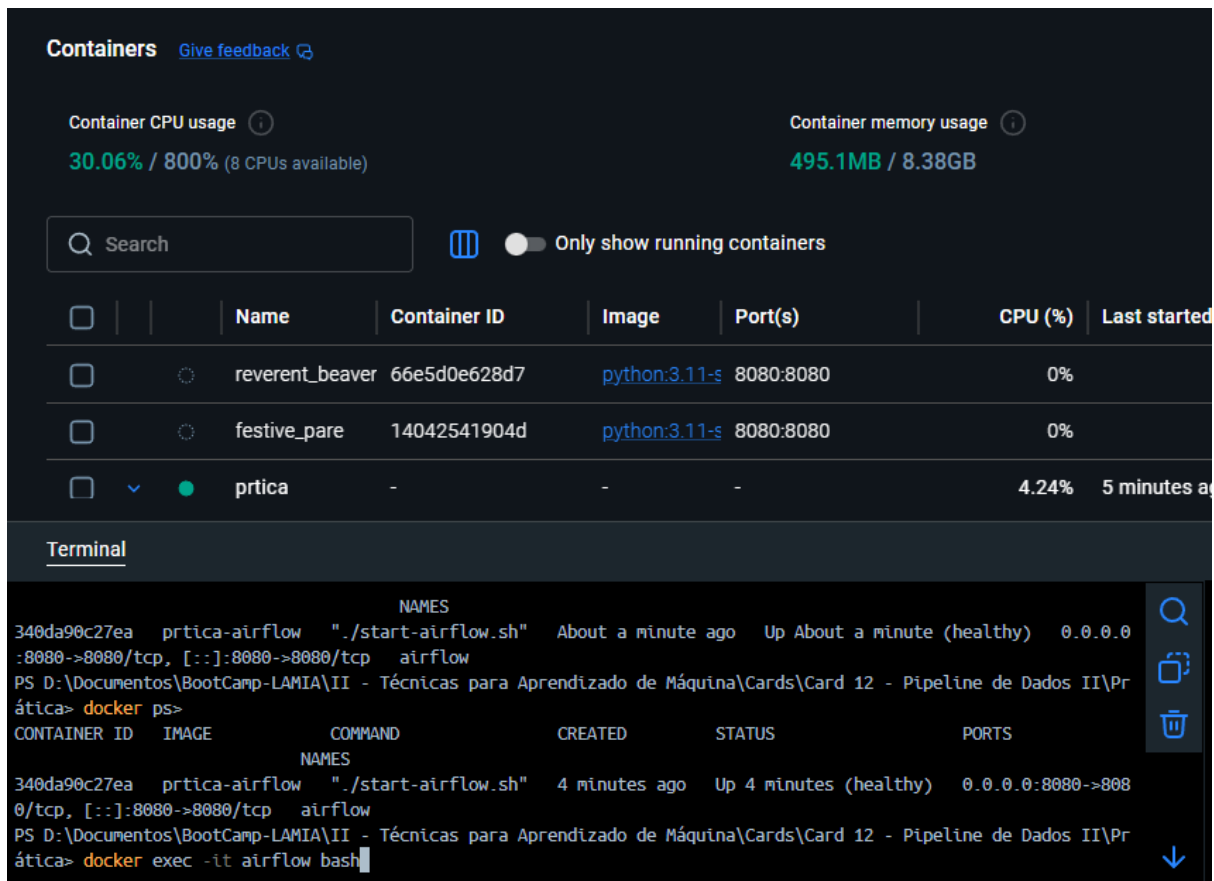


Figura 4. Verificando os contêineres e entrando no que acabei de criar. (autoria própria)

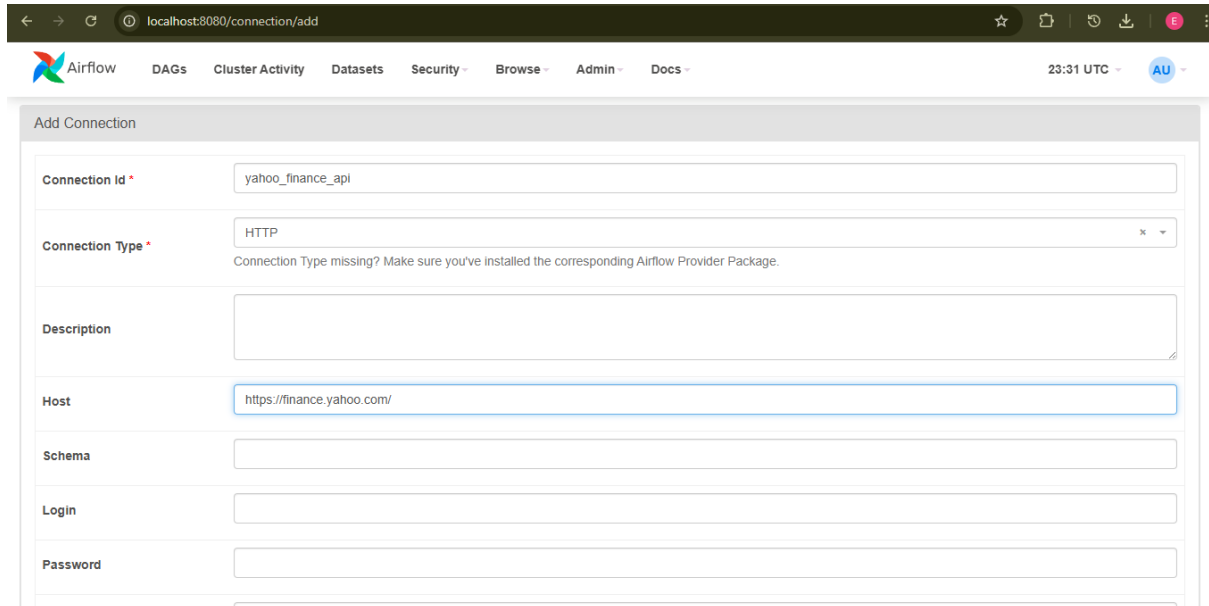


Figura 5. Criando a conexão da API do Yahoo Finance no Airflow. (autoria própria)

Para enviar o a mensagem no *Discord*, era necessário criar um Webhook e copiar o *link* disponibilizado pelo *Discord*, para isso, no *Discord*, segui os seguintes passos: configurações do canal (criei um canal apenas para isso) → Integrações → *Webhooks* → Novo *webhook*. Atribui um nome novo a ele, copiei a URL dele e salvei.

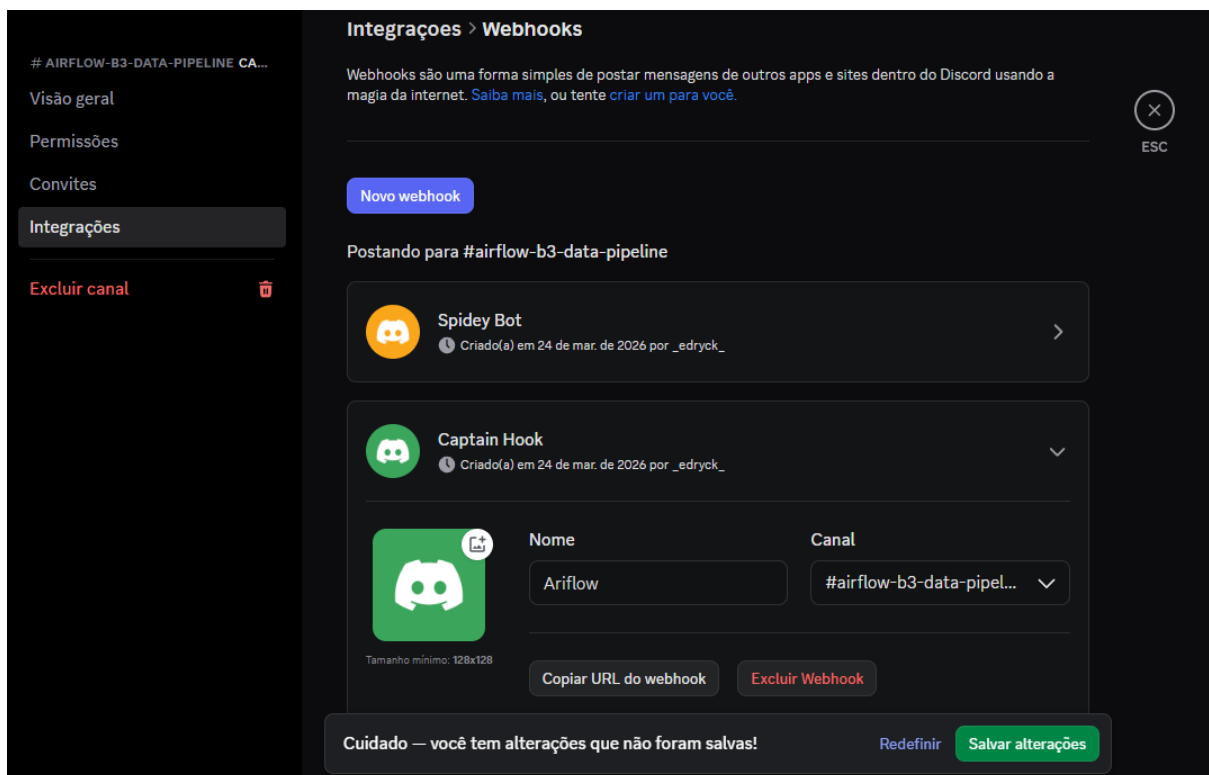
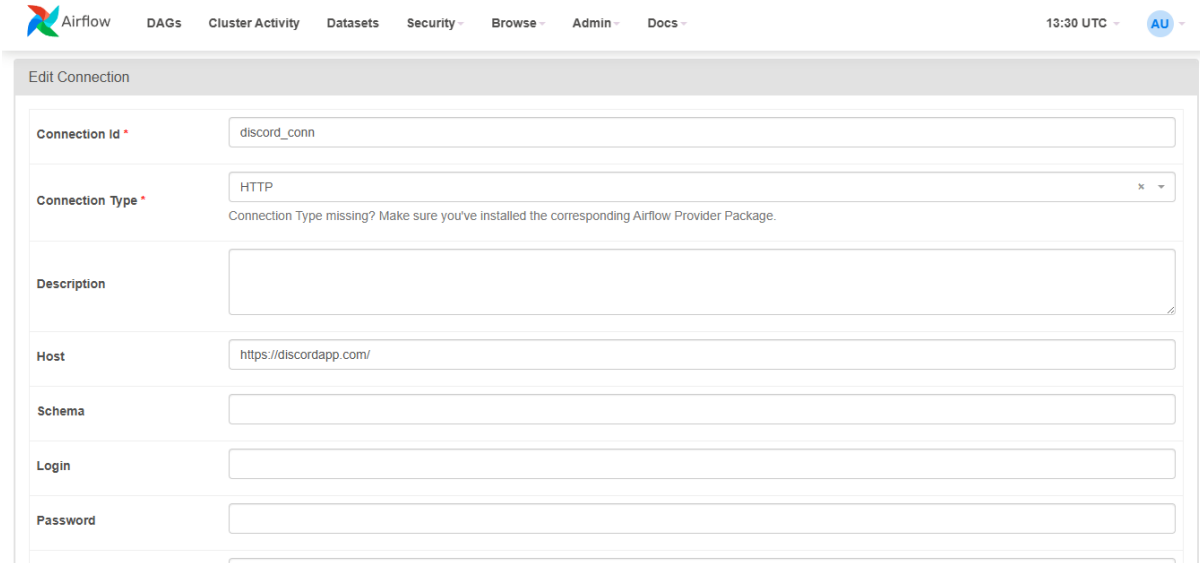


Figura 6. *Webhook*. (autoria própria)

O *link* copiado é dividi da seguinte forma: URL/api/webhooks/ID/TOKEN. Exemplo:

`https://discordapp.com/api/webhooks/<id-do-webhook>/<token-do-webhook>`

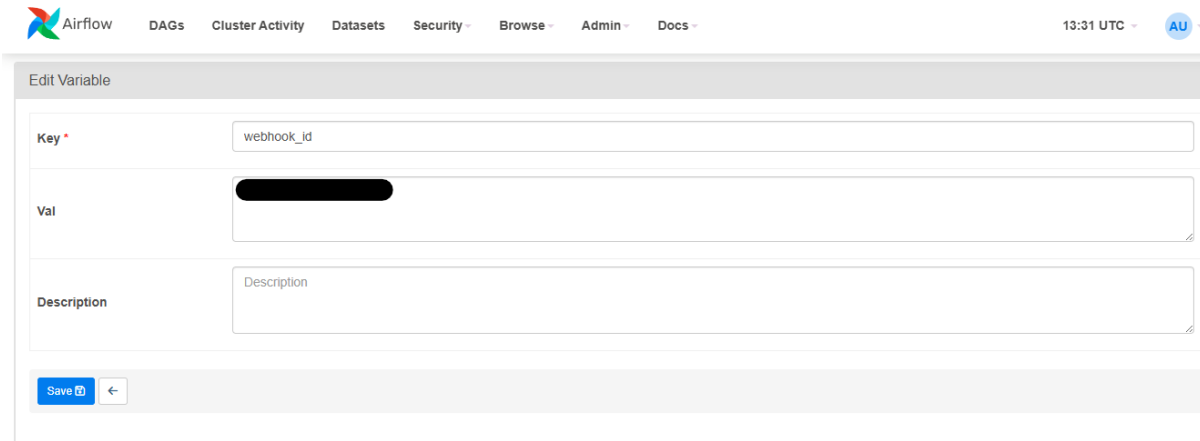
No Airflow, precisa criar a conexão e duas variáveis, a conexão é do tipo HTTP e as Variáveis são o Id e o Token.



The screenshot shows the 'Edit Connection' form in the Airflow web interface. The form fields are as follows:

- Connection Id ***: discord_conn
- Connection Type ***: HTTP (with a dropdown arrow and a message: 'Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.')
- Description**: (empty text area)
- Host**: https://discordapp.com/
- Schema**: (empty text field)
- Login**: (empty text field)
- Password**: (empty text field)

Figura 7. Criando a conexão. (autoria própria)

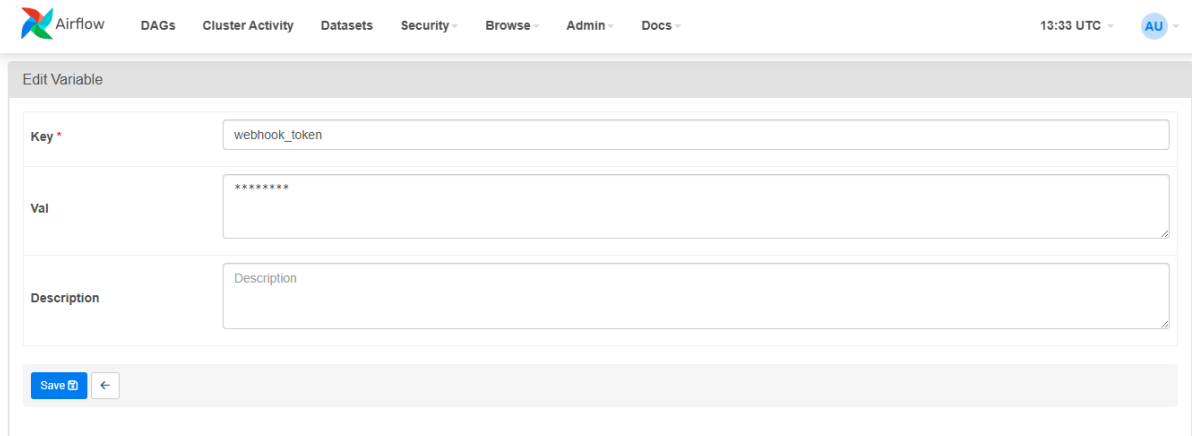


The screenshot shows the 'Edit Variable' form in the Airflow web interface. The form fields are as follows:

- Key ***: webhook_id
- Val**: (redacted with a black bar)
- Description**: (empty text area)

At the bottom of the form, there is a 'Save' button and a back arrow button.

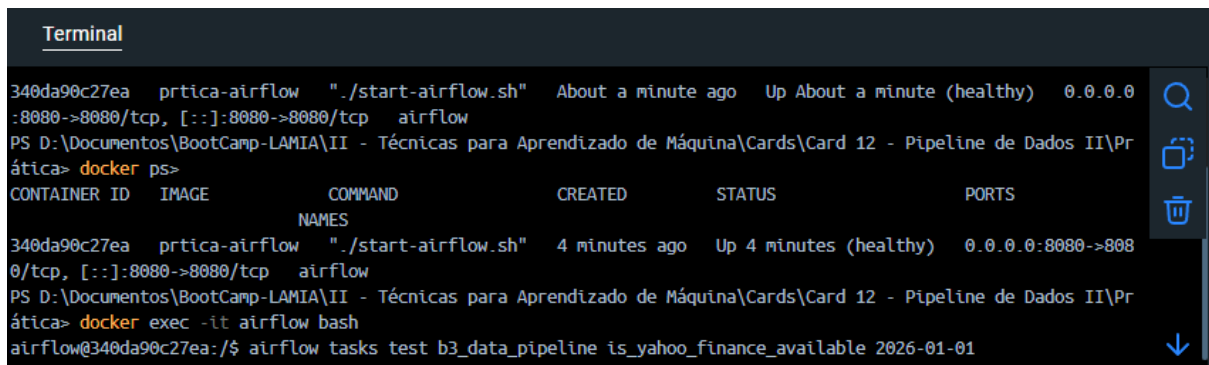
Figura 8. Criando a variável do ID. (autoria própria)



The image shows the 'Edit Variable' form in the Apache Airflow web interface. The form has three main sections: 'Key', 'Val', and 'Description'. The 'Key' field contains 'webhook_token'. The 'Val' field contains a masked value '*****'. The 'Description' field is empty. At the bottom, there is a 'Save' button and a back arrow.

Figura 9. Adicionando a variável com o token. (autoria própria)

Agora que o código estava pronto, as conexões criadas e as variáveis também, era só testar as *tasks* e colocar para rodar a DAG.



The terminal shows the command `docker ps` being executed, displaying the status of the `prtica-airflow` container. The container is running and healthy. Below the command output, the command `airflow tasks test b3_data_pipeline is_yahoo_finance_available 2026-01-01` is shown.

Figura 10. Testando as *tasks*. (autoria própria)



The terminal shows the output of the `generate_summary` task. It displays a list of stock prices and their percentage changes. The output is as follows:

Stock	Price	Change
WEGE3.SA	R\$50.08	▲1.03%
ABEV3.SA	R\$15.13	▲0.87%
VALE3.SA	R\$81.34	▲0.74%
ITUB4.SA	R\$42.40	▲0.26%
PETR4.SA	R\$50.17	▲0.2%
BBAS3.SA	R\$22.74	▲0.09%
BBDC4.SA	R\$18.80	▼0.0%

Summary:
 Maior alta: WEGE3.SA (1.03%)
 Maior queda: BBDC4.SA (0.0%)

Figura 11. Resultado da *task* de gerar o resumo do dia¹. (autoria própria)

¹ OBS.: Esse resumo aparece no terminal ao executar o teste da *task* `generate_summary`.

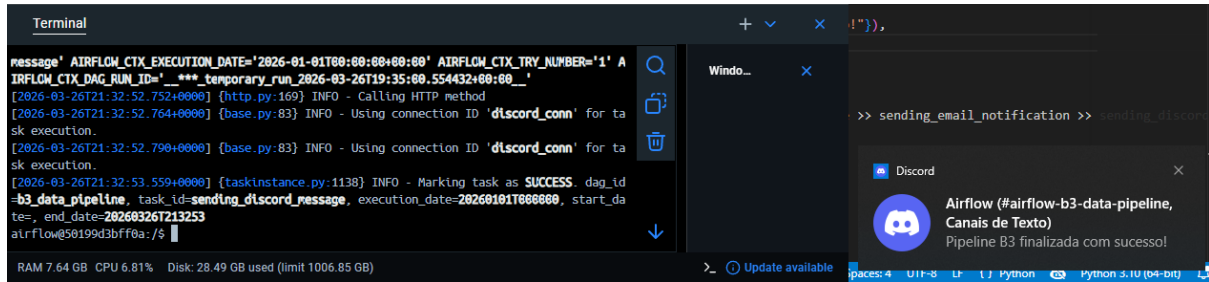


Figura 12. Resultado da última *task*, a de mandar a mensagem no *Discord*. (autoria própria)

Depois de testada, só ativar no *Airflow*.

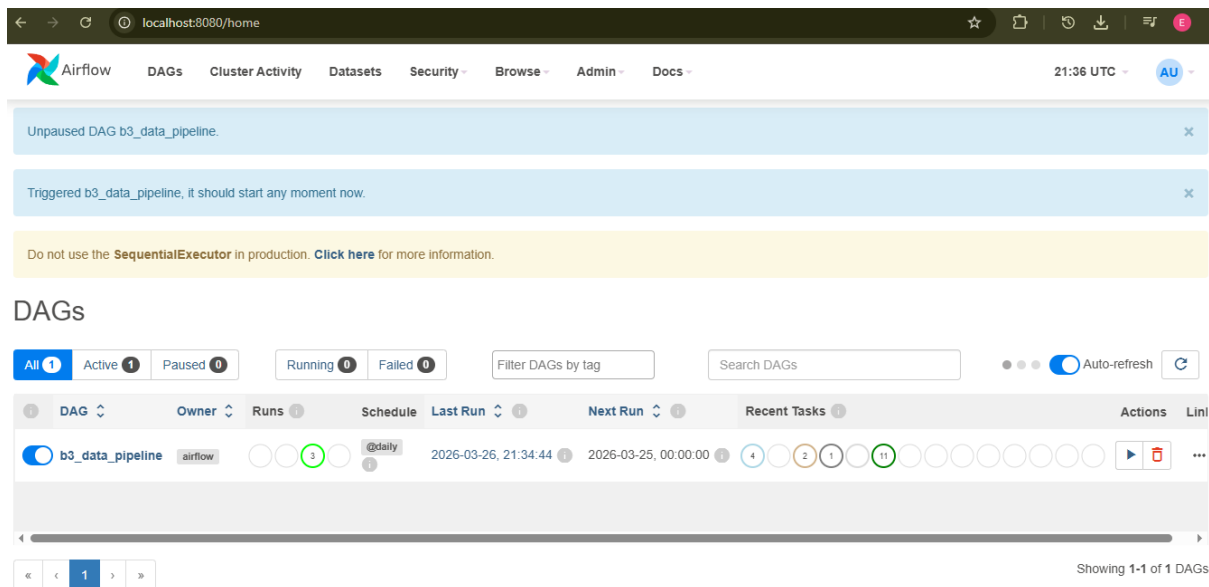


Figura 13. *DAG* sendo executada. (autoria própria)

3. Conclusão

A criação de uma *pipeline* de dados automatizada é algo que essencial para aumentar a eficiência da extração, preparação e armazenamento dos dados que posteriormente serão utilizados treinar um modelo. O *Docker* garante a execução da *DAG* em qualquer máquina. Tive dificuldades neste *Card* para executar os códigos da aula, por estar bem desatualizada, e encontrar o módulo certo na documentação do *Airflow*. Finalmente, a *pipeline* está pronta, toda vez que ela finalizar (diariamente) será enviada uma mensagem no *Discord* e um email.